Rutuja Bhalekar D20A - 03

EXPERIMENT 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Implementation and Output:

```
    # Get the previous block
    def get_previous_block(self):
        return self.chain[-1]

    # Create a transaction
    def create_transaction(self, sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)
        return transaction

    # Proof of Work
    def proof_of_work(self, previous_proof):

        new_proof = 1

        while True:

            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:3] == '000':
                return new_proof
```

```
            new_proof += 1

    # Hash a block
    def hash(self, block):

        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()

        return hashlib.sha256(encoded_block).hexdigest()

    # Check blockchain validity
    def is_chain_valid(self, chain):

        previous_block = chain[0]
        block_index = 1

        while block_index < len(chain):

            block = chain[block_index]

            # Check previous hash
            if block['previous_hash'] != self.hash(previous_block):
                return False

            # Check Proof of Work
            previous_proof = previous_block['proof']
            proof = block['proof']
```

```
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:3] != '000':
                return False

            previous_block = block
            block_index += 1

        return True


# Creating the Blockchain
blockchain = Blockchain()

print("Genesis Block:")
print(blockchain.chain)
```

```
Genesis Block:
[{'index': 1, 'timestamp': '2026-09-02 06:28:20.631138', 'proof': 1, 'previous_hash': '0', 'transactions': []}]
```

```
sender = input("Enter sender: ")
receiver = input("Enter receiver: ")
amount = input("Enter amount: ")

transaction = blockchain.create_transaction(
    sender,
    receiver,
    amount
)

print("\nTransaction added successfully:")
print(transaction)
```

```
Enter sender: Rutuja
Enter receiver: Isha
Enter amount: 10000

Transaction added successfully:
{'sender': 'Rutuja', 'receiver': 'Isha', 'amount': '10000'}
```

```python
def mine_block():
    # Check whether transactions are available
    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions available. Block cannot be mined.'}
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    # Find Proof of Work
    proof = blockchain.proof_of_work(previous_proof)
    # Get previous block hash
    previous_hash = blockchain.hash(previous_block)
    # Store current transactions
    transactions = blockchain.transactions.copy()
    # Clear pending transactions
    blockchain.transactions.clear()
    # Create a new block
    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions
    )
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']}
    return response
```

```python
print(mine_block())
```

```
{'message': 'Congratulations, you just mined a block!', 'index': 2, 'timestamp': '2026-09-02 06:47:46.592037', 'proof': 533,
```

```
'previous_hash': '6540369e1bc0b83807e46c56a9496756400f8775c9ca1ab2bbd0edd00466713f', 'transactions': [{'sender': 'Rutuja', 'receiver': 'Isha', 'amount': '10000'}]}
```

```python
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain),
        'pending_transactions': blockchain.transactions
    }

    return response


print(get_chain())
```

```
{'chain': [{'index': 1, 'timestamp': '2026-09-02 06:28:20.631138',
```

```python
def is_valid():

    valid = blockchain.is_chain_valid(blockchain.chain)

    if valid:

        response = {
            'message': 'All good. The Blockchain is valid.'
        }

    else:

        response = {
            'message': 'Houston, we have a problem. The Blockchain is not valid.'
        }

    return response


print(is_valid())
```

```
{'message': 'All good. The Blockchain is valid.'}
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.